

Opinion vs. Fact Classification Using NLP

Natural Language Processing VU (DAT.C309UF) · SS26

Group 51 · Graz University of Technology

Abstract

This report presents a two-stage NLP pipeline for classifying short text segments as either *Fact* or *Opinion*. Stage 1 employs traditional machine learning with hand-crafted linguistic features — including TF-IDF, part-of-speech (POS) ratios, named entity recognition (NER), and a keyword-based knowledge base — combined with Logistic Regression. The best Stage 1 model (KB + TF-IDF + POS + NER) achieves a macro F1-score of 0.877 on a held-out test set. Stage 2 uses the Llama-3.1-8B-Instant model via the Groq API and systematically evaluates six prompting strategies: Zero-Shot, Few-Shot, RAG, RAG + Few-Shot, Self-Consistency, and their full combination. The combined RAG + Few-Shot + Self-Consistency approach delivers the best Stage 2 result with a macro F1-score of 0.959. Both pipelines are then applied to a held-out validation set of 222 samples, for which predictions are submitted as required.

1. Introduction

Distinguishing factual statements from subjective opinions is a fundamental task in natural language understanding with applications spanning journalism, social media moderation, and automated fact-checking. A *fact* is an objectively verifiable claim, often supported by data, citations, or established evidence, whereas an *opinion* expresses a personal belief, judgment, or attitude that cannot be universally verified.

This project is part of the NLP VU practicals for SS26 at TU Graz. We are given a labeled dataset of 855 sentences (443 facts, 412 opinions) and a validation set of 222 unlabeled entries. The task requires building two independent pipelines: one based on classical NLP and supervised learning (Stage 1), and one leveraging a large language model (LLM) with advanced prompting strategies (Stage 2). Both must be clearly separated and each must include a baseline for comparison.

2. Related Work

Classical approaches to fact/opinion classification rely on surface-level lexical features. Wiebe et al. (2004) showed that subjectivity in text correlates strongly with certain POS patterns and sentiment-bearing lexica. TF-IDF representations have long served as strong baselines for text classification tasks due to their ability to capture discriminative n-grams. More recently, Retrieval-Augmented Generation (RAG) — introduced by Lewis et al. (2020) — demonstrated that grounding LLM generation in external knowledge substantially improves factual accuracy. Self-Consistency prompting (Wang et al., 2022) further improves LLM reliability by sampling multiple reasoning paths and taking a majority vote, reducing variance introduced by the model's stochastic outputs.

3. Materials and Methods

3.1 Dataset

The training dataset consists of 855 labeled text entries: 443 facts (51.8%) and 412 opinions (48.2%), sourced from a variety of domains. An 80/20 stratified train-test split was applied ($X_{\text{train}} = 684$, $X_{\text{test}} = 171$), ensuring equal class representation in both splits. The validation set contains 222 unlabeled entries released one week before the deadline.

3.2 Stage 1 — Traditional NLP

Stage 1 builds a feature matrix from four complementary representations and trains a Logistic Regression classifier on their combination.

Knowledge Base (KB). We manually compiled two keyword lists: an *opinion marker* list (e.g., *i think*, *i believe*, *probably*, *ridiculous*) and a *fact marker* list (e.g., *according to*, *researchers*, *published*, *percent*). Each

text is represented by binary indicator features for every keyword in both lists, forming a 21-dimensional sparse vector.

TF-IDF. A unigram TF-IDF vectorizer with a vocabulary cap of 5,000 terms is fitted on the training split and applied to the test split. TF-IDF serves as the primary lexical signal and our main content-based feature.

POS Ratios. Using spaCy's *en_core_web_sm* pipeline, each text is parsed and the proportion of tokens belonging to each universal POS tag (NOUN, VERB, ADJ, ADV, etc.) is computed. Opinions tend to be richer in adjectives and first-person pronouns, while facts lean towards nouns and proper names.

NER Ratios. Named entity labels (PERSON, ORG, DATE, GPE, etc.) are extracted and normalised by text length. Factual texts tend to contain denser entity annotations, particularly organisations, dates, and numeric quantities.

Baseline. A rule-based classifier assigns *opinion* when opinion keyword count exceeds fact keyword count, and *fact* otherwise. This KB-only baseline achieved an accuracy of 56.14% and a macro F1 of 0.446, demonstrating that keywords alone are insufficient and confirming the need for richer feature combinations.

Model. All feature matrices are horizontally stacked using scipy's *hstack* and passed to a Logistic Regression classifier with a maximum of 1,500 iterations. We evaluate four combinations in order of increasing complexity.

3.3 Stage 2 — LLM-Based Classification

Stage 2 uses **Llama-3.1-8B-Instant** served via the **Groq API** with the OpenAI-compatible client. Temperature is fixed at 0 to ensure deterministic outputs for all single-pass methods; the same 80/20 split ($X_{\text{test}} = 171$) is used for evaluation before applying the best method to the full validation set.

I. Zero-Shot Learning (Baseline). The LLM is given a minimal system prompt defining fact and opinion and asked to return a single word. This serves as the Stage 2 baseline.

II. Few-Shot Learning. Four curated examples (two facts, two opinions) are prepended to the prompt to anchor the model's decision boundary with concrete instances.

III. RAG. Before each LLM call, the input text is matched against our knowledge base keyword lists. Matched keywords are injected into the prompt as 'Knowledge Base hints', providing lightweight retrieval-augmented context without a vector database.

IV. RAG + Few-Shot. The KB hints and the four curated examples are combined in a single prompt, allowing the model to cross-reference both structured hints and labelled exemplars.

V. Self-Consistency. The zero-shot prompt is run three times per input with temperature 0; the majority vote across the three outputs is taken as the final label. This reduces the effect of occasional model inconsistencies.

VI. RAG + Few-Shot + Self-Consistency (Best). The full combined prompt (KB hints + few-shot examples) is run three times per input, and a majority vote is taken. This is our most sophisticated method and yielded the highest performance.

4. Results

4.1 Stage 1 Results

Table 1 summarises the performance of all Stage 1 configurations on the held-out test set ($n = 171$).

Model	Accuracy	Macro F1
KB Rule-Based Baseline	56.14%	0.446
KB + TF-IDF	85.96%	0.859
KB + TF-IDF + POS	87.13%	0.871
KB + TF-IDF + NER	85.96%	0.859
KB + TF-IDF + POS + NER ★	87.72%	0.877

Table 1: Stage 1 model comparison on the test set (n=171). ★ = best model.

Each feature group adds meaningful signal. Adding POS ratios to KB + TF-IDF raises the F1 from 0.859 to 0.871 (+0.012), confirming that part-of-speech distributions carry discriminative power for subjectivity detection. NER alone provides a smaller benefit than POS, but combining all four feature types yields the best result: 87.72% accuracy and F1 = 0.877. This model was applied to the validation set to generate *group51_classifications_1.csv*.

4.2 Stage 2 Results

Table 2 compares all six Stage 2 methods on the same test set (n = 171). Zero-Shot Learning serves as the Stage 2 baseline.

Method	Accuracy	Macro F1
I. Zero-Shot (Baseline)	91.23%	0.912
II. Few-Shot	87.72%	0.877
III. RAG	92.40%	0.924
IV. RAG + Few-Shot	92.98%	0.930
V. Self-Consistency	91.23%	0.912
VI. RAG + Few-Shot + Self-Consistency ★	95.91%	0.959

Table 2: Stage 2 prompting method comparison on the test set (n=171). ★ = best method.

Several findings are notable. First, Few-Shot (II) actually performs *worse* than Zero-Shot (I) in terms of F1 (0.877 vs 0.912), despite the added examples. Inspection of the classification report reveals that Few-Shot inflates fact-precision (0.97) at the cost of recall (0.79), suggesting the examples biased the model toward conservatively labelling ambiguous texts as facts. RAG (III) corrects this by providing contextual keyword hints, boosting F1 to 0.924. Combining RAG with Few-Shot examples (IV) further improves both precision and recall symmetrically (F1 = 0.930). Self-Consistency alone (V) matches the Zero-Shot baseline exactly, indicating that three runs of the same minimal prompt converge on the same boundary. The full combination (VI) achieves the best result: 95.91% accuracy and F1 = 0.959, with near-equal precision and recall for both classes (both 0.93–0.99 range).

4.3 Validation Set Predictions

The best Stage 2 method (RAG + Few-Shot + Self-Consistency) was automatically selected and applied to all 222 validation samples. The resulting distribution is 108 Facts (48.6%) and 114 Opinions (51.4%), closely mirroring the class balance in the training data. Predictions were saved to *group51_classifications_2.csv*.

5. Discussion

5.1 Stage 1 Analysis

The KB-only baseline performs only slightly above chance (56%), which is expected — keyword matching is a brittle heuristic that ignores word context entirely. The jump to 85.96% upon adding TF-IDF confirms that discriminative vocabulary patterns are the primary signal. POS ratios provide consistent additive gains: adjective and adverb density are reliable proxies for subjective language, consistent with prior subjectivity

detection literature. NER features add less marginal value than POS, likely because the dataset's opinion sentences can also contain named entities (e.g., critiques of specific people or organisations). The final model (F1 = 0.877) is competitive given the constraint of avoiding transformers in Stage 1.

5.2 Stage 2 Analysis

The progression from Zero-Shot to the full RAG + Few-Shot + Self-Consistency combination illustrates how each prompting technique addresses a different failure mode. Zero-Shot is already a strong baseline (F1 = 0.912) because Llama-3.1 has substantial world knowledge about language subjectivity. Few-Shot alone fails to improve matters, suggesting that four examples are insufficient to overcome the model's inherent biases for this task without supporting context. RAG, even in our lightweight keyword-injection form, provides the external grounding needed to push performance above Zero-Shot. Self-Consistency's benefit materialises only when the underlying prompt is already strong (VI vs V), implying that voting on poor individual predictions does not help. The final method's symmetric per-class performance (both 0.96 F1) confirms that the combination successfully balances precision and recall across both classes.

5.3 Limitations

Our RAG implementation is lightweight (keyword lookup) rather than a proper vector retrieval system, which limits its ability to surface contextually distant but semantically relevant knowledge. Self-Consistency with only three runs may not fully stabilise the vote for borderline cases; increasing to five or more runs could help but at higher API cost. The Stage 1 model is English-only and lacks handling for code-switched or multilingual inputs. Finally, all Stage 2 evaluations depend on the Groq API rate limits and model availability, which required splitting work across two API keys.

6. Conclusion

We implemented and evaluated two independent pipelines for opinion vs. fact classification. In Stage 1, combining a handcrafted knowledge base with TF-IDF, POS, and NER features — all fed into Logistic Regression — achieves a macro F1 of 0.877, substantially above the keyword-only baseline of 0.446. In Stage 2, systematically stacking Zero-Shot, RAG, Few-Shot, and Self-Consistency prompting over Llama-3.1-8B-Instant raises the macro F1 from 0.912 (Zero-Shot baseline) to 0.959 (full combination), showing that each technique addresses a distinct weakness of the others. The LLM-based pipeline outperforms the traditional one by roughly 8 F1 points, but comes at the cost of API latency, rate-limit management, and the need for two Groq API keys. Future work could explore a dense-retrieval RAG with a proper vector store, Chain-of-Thought prompting, and PEFT fine-tuning on the training set.

References

Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 33.

Wang, X. et al. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171.

Wiebe, J., Wilson, T., & Cardie, C. (2004). Annotating Expressions of Opinions and Emotions in Language. Language Resources and Evaluation, 39(2-3), 165–210.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830.

Honnibal, M. & Montani, I. (2017). spaCy 2: Natural Language Understanding with Bloom Embeddings, CNNs and Rank-Based Models.

Appendix — Use of AI

We used ChatGPT (OpenAI) to assist with initial literature search and to suggest suitable prompting strategies for Stage 2. All feature engineering, code implementation, and final design decisions were made independently. No data was classified by AI directly.

Contributions

Task	Member A	Member B
------	----------	----------

Dataset analysis & split	50%	50%
Stage 1: Feature engineering	60%	40%
Stage 1: Model training & eval	50%	50%
Stage 2: Prompting strategies	40%	60%
Stage 2: Evaluation & validation	50%	50%
Report writing	50%	50%